STA314, Lecture 7

November 9, 2021
Stanislav Volgushev

Dimension reduction methods

General idea of dimension reduction methods

Prediction perspective: all methods we discussed so far tried to balance bias vs variance.

- Model selection (AIC, BIC etc): reduce variance by discarding predictors that are not important.
- Ridge and Lasso: reduce variance by adding a penalty term (lasso can also be used to perform variable selection).

Next: alternative methods based on *dimension reduction*

- Model selection: only keep 'important' predictors.
- Dimension reduction: transform predictors to only keep the 'important properties' of original predictors.

Dimension reduction methods: basic set-up (linear case)

Data: (X_i, y_i) with predictors $X_i = (x_{i,1}, ..., x_{i,p})^\top \in \mathbb{R}^p$.

Step 1: define standardized predictors $\tilde{X}_i = (\tilde{x}_{i,1}, ..., \tilde{x}_{i,p}^\top)$ with $\tilde{x}_{i,k} = x_{i,k} - \bar{x}_{\cdot,k}$.

Step 2: form new predictors $Z_i = (z_{i,1}, ..., z_{i,M})^\top \in \mathbb{R}^M$ by taking linear combinations of original predictors, i.e.

$$z_{i,m} = \sum_{k=1}^{p} \phi_{k,m} \tilde{x}_{i,k}, \quad m = 1, ..., M$$

where $\phi_{k,m}$ are some weights determined later.

Step 3: run a linear regression using Z_i as predictors.

- If $M < p$, the model described above is less flexible compared to full linear model, thus variance can be reduced. Might introduce bias.
- There are several ways to select the weights $\phi_{k,\cdot}$. Two popular approaches mentioned today: **P**rincipal **C**omponent **R**egression and **P**artial **L**east **S**quares.

Recap: projections on different directions in $\mathbb{R}^p$

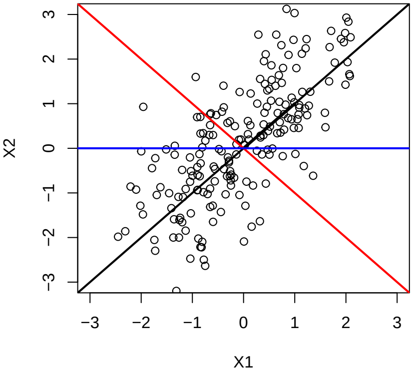
Note: if $\sum_{k=1}^{p} \phi_{k,m}^2 = 1$, then $\Phi_m := (\phi_{1,m}, ..., \phi_{p,m})^\top$ describes direction in $\mathbb{R}^p$.

$\sum_{k=1}^{p} \phi_{k,m} \tilde{x}_{i,k} = \Phi_m^\top \tilde{X}_i$ corresponds to length of X_i when projected on that direction.

Dimension reduction methods: Principal Component regression

PCR: form linear combinations that explain 'the most variation' in the predictors.

First 'principal component': $z_{i,1} = \sum_{k=1}^{p} \phi_{k,1} x_{i,k}$ describes the direction in $\mathbb{R}^p$ along that predictors show 'maximal variation'. Figure below: which line corresponds to first principal component?



Principal Component Regression

Mathematical definition of first principal component:

$$\Phi_1^{PCR} = \arg \max_{V \in \mathbb{R}^p \sum v_k^2 = 1} \frac{1}{n-1} \sum_{i=1}^{n} \left(\sum_{k=1}^{p} v_k \tilde{x}_{i,k} \right)^2$$
$$= \arg \max_{V \in \mathbb{R}^p : \sum v_k^2 = 1} \frac{1}{n-1} \sum_{i=1}^{n} (V^\top \tilde{X}_i)^2.$$

- For any given vector V the term $V^\top \tilde{X}_i$ is a candidate predictor.
- $\sum v_k^2 = 1$ ensures that V is a direction (vector V has norm one).
- Since $\sum_i V^\top \tilde{X}_i = 0$, the sum above is the sample variance of $V^\top \tilde{X}_1, ..., V^\top \tilde{X}_n$

The first principal component corresponds to the linear combination of predictors that has the largest sample variance.

Note: if the multiplicity of the largest eigenvalue of $\tilde{\mathrm{X}}^\top \tilde{\mathrm{X}}$ is one, this is unique up to sign. Otherwise there are many solutions.

Principal Component regression

After we explained as much variation as we can by picking one direction, there is still some variation in the other directions. That variation is explained the next principal components.

The second principal component corresponds to the direction with largest variance which is orthogonal to the first component.

$$\Phi_2^{PCR} = \arg \max_{V \in \mathbb{R}^p : \sum v_k^2 = 1, V^\top \Phi_1^{PCR} = 0} \frac{1}{n-1} \sum_{i=1}^{n} (V^\top \tilde{X}_i)^2.$$

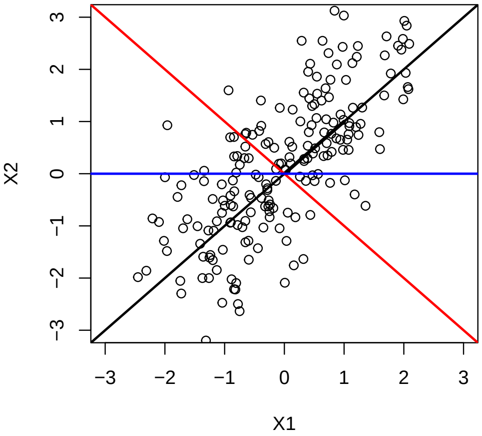
The third principal component corresponds to the direction with largest variance which is orthogonal to the first two principal components. Exercise: write down formula.

Subsequent principal components defined similarly.

How many principal components can we have at most if $X_i \in \mathbb{R}^p$?

Exercise: describe principal directions through eigenvectors of $\tilde{\mathrm{X}}^\top \tilde{\mathrm{X}}$.

Where is the second principal component?



Comments on principal component regression

- Despite the fact PCR with $M < p$ leads to less flexible models, it does not lead to the selection of relevant predictors since each principal component can depend on all predictors.
- The number of components is usually determined by cross-validation.
- How well PCR works depends on how well the principal components can help to predict the response.
- When applying PCR, the predictors are usually standardised to have not only sample mean zero but also sample variance 1.

Note: the idea of using directions of maximal variation is sometimes also used to simply describe structure in a data set without a response. This is called *Principal Component Analysis* and is an example of *unsupervised learning*.

Partial least squares

- Trouble with PCR: does not take into account any information about the response. It can happen that directions of high variation in X_i are not useful for predicting y_i.
- Partial Least Squares: determine directions in a way that maximises their power for predicting response.
- Details omitted here, see pp 80-82 and Algorithm 3.3 in 'Elements of Statistical Learning' for detailed description.
- We will see some `R` examples.

R examples: `STA314-2021-Lecture07-DimensionReduction`

The high-dimensional setting $n < p$

The high-dimensional setting

In Statistics, we talk about the 'high-dimensional' setting when the number of predictors p is of comparable order or even larger than the sample size n (sometimes the latter is called ultra high dimensional).

Examples:

1. Genetics: predict risk of certain type of cancer from DNA mutations. Usually number of objects n is in the hundreds, but roughly $p = 500000$ common DNA mutations.
2. Marketing: for each customer in an online shop, have 0-1 encoding of items customer decided to buy or not to buy. Predict future shopping behaviour based on past purchase.

Clearly k-nn won't work in this setting. Which of the methods for linear regression discussed so far could be applicable?

'Usual' linear regression when $p > n$

Usual linear regression is not applicable when $p > n$ because there will be no unique solution to OLS problem. Recall: there is a unique minimizer of

$$RSS(b) = \sum_{i=1}^{n} (y_i - X_i^\top b)^2$$

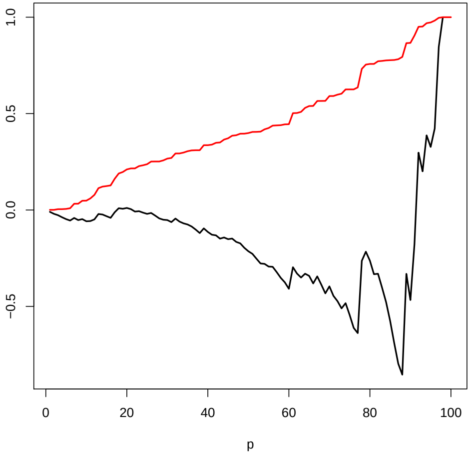
if and only if matrix $\mathrm{X}^\top \mathrm{X}$ is invertible. Matrix $\mathrm{X}^\top \mathrm{X}$ is ...-dimensional and has rank at most .... If $p > n$ it can not be invertible.

Even if p slightly smaller than n, linear regression will give a very small training error even if none of the predictors are relevant for response. Resulting test error will be very large.

Since usual least squares is not applicable, we need to look for *less flexible* models.

Adjusted R^2 fails for high dimensions

Setting: generate p predictors not related to the response, run linear regression. Sample size: $n = 100$. Plot below: R^2 and adjusted R^2 as a function of p. Which one is which? *R Exercise: generate this type of plot.*



Classical model selection methods

Backward stepwise selection: need to start with model that has all predictors, does not make sense when $p > n$.

Forward stepwise selection: can be used to select models as long as the largest model has less predictors than observations. So forward stepwise selection can be applied, but we need to stop before too many predictors are selected.

Best subset: often too expensive from a computational point of view. If there are p predictors, there are p choose k models with k predictors, that grows in p like p^k. If p, k are large this is not feasible computationally.

How do we compare models with different numbers of predictors?

- adjusted R^2 does not work for p close to n.
- Trouble with C_p, AIC, BIC: need $\hat{\sigma}^2$, difficult to obtain if $p > n$. The usual way to get this from 'full model' does not work, so not easily applicable when $p > n$.
- Cross-validation can be applied, but can also be expensive computationally.

Lasso, Ridge, PCR, PLS

Lasso: can often be applied even if $p > n$, provided λ is selected by cross-validation. In fact, one of the reasons for the great success of Lasso is the increased importance of data sets with $p > n$.

Ridge: will always lead to a unique solution if $\lambda > 0$, so can be applied when $p > n$. Disadvantage compared to Lasso: does not set any coefficients to zero, so can not be used to perform model selection.

PCR, PLS: can be applied if $p > n$ but $M < n$ components are used, number of components can be selected by cross-validation.

Moving away from linearity

Moving away from linearity

In the last couple of lectures, we came up with different ways to build models that are *less flexible* compared to linear models in order to reduce the variance.

The main motivation was trading bias for variance.

From now on: look for models that are *more flexible* than just usual linear models.
Motivation: try to model non-linear influence of predictor on outcome (for example through non-linear transformations of predictors) in order to decrease bias.

We will cover (briefly)

- *Basis expansions* (one predictor): popular classes of non-linear transformations of original predictor.
- *Smoothing splines*: similar idea to Basis expansion, but without some of the drawbacks.
- *Local regression*: generalize idea behind k-nn.
- *Generalized additive models (GAM)*: combining ideas mentioned above with several predictors.

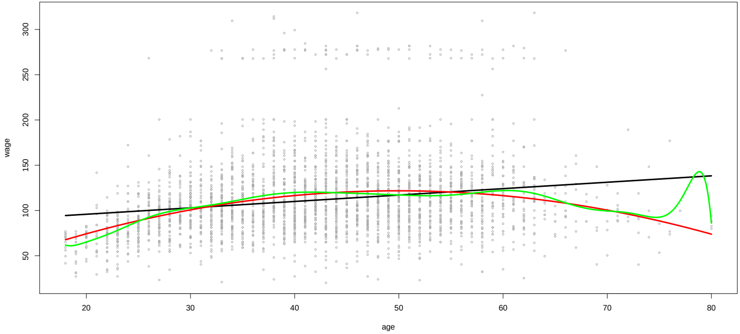
Non-linear transformations: polynomials

Have already seen polynomials earlier. Polynomial of degree d takes the form

$$f(x) = b_1 + b_2 x + b_3 x^2 + \ldots + b_{d+1} x^d, \quad b_{d+1} \neq 0.$$

Model can be learned using a linear model with new predictors, $x, x^2, \ldots, x^d$.

+ Simple to fit, can model non-linear relationships.
– Put a 'global' structure on the function f. Can be very wild near boundary points. Example below: degree 1 (black), 2 (red) and 15 (green).



Non-linear transformations: step functions

1. Divide range of predictor x into consecutive intervals $(c_1, c_2], (c_2, c_3], ..., (c_{K-1}, c_K]$ with each interval containing some values from the sample $x_1, ..., x_n$.
2. For $k = 1, ..., K-1$, let[1]

$$\hat{\mu}_k := \frac{\sum_{i=1}^n y_i I\{x_i \in (c_k, c_{k+1}]\}}{\sum_{i=1}^n I\{x_i \in (c_k, c_{k+1}]\}}.$$

3. The predicted values are

$$\hat{f}(x) := \sum_{k=1}^{K-1} \hat{\mu}_k I\{x \in (c_k, c_{k+1}]\}.$$

- In words: predicted value for $x \in (c_j, c_{j+1}]$ is average of all y_i with $x_i \in (c_j, c_{j+1}]$.

+ Simple to fit and interpret.
– Have discontinuities at breakpoints.
– Need to select breakpoints (number and location).

[1] here $I\{x_i \in (c_k, c_{k+1}]\} = 1$ if $x_i \in (c_k, c_{k+1}]$ and 0 else.

Non-linear transformations: piecewise polynomial regression

1. Divide range of predictor x into consecutive intervals $(c_1, c_2], \ldots, (c_{K-1}, c_K]$ with each interval containing some values from the sample $x_1, \ldots, x_n$.
2. For $k = 1, \ldots, K-1$, let

$$\hat{B}_k := \arg \min_{B=(b_1,\ldots,b_{d+1})\in\mathbb{R}^{d+1}} \sum_{i:x_i\in(c_k,c_{k+1}]} (y_i - b_1 - b_2 x_i - \ldots - b_{d+1}x_i^d)^2$$

3. The predicted values are

$$\hat{f}(x) := \sum_{k=1}^{K-1} I\{x \in (c_k, c_{k+1}]\}(1, x, \ldots, x^d)\hat{B}_k$$

+ Simple to fit and interpret.
+ More flexible compared to step functions since shape between breakpoints is less constrained. Can require less breakpoints compared to step functions.

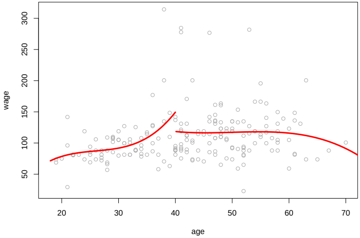
– Have jumps at breakpoints, need to manually select breakpoints.

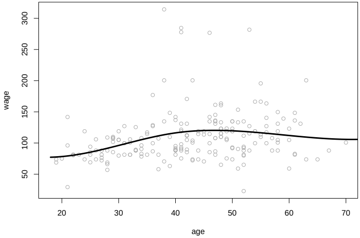
Polynomial splines and natural polynomial splines

One of the dis-advantages of fitting piecewise polynomials: discontinuities at breakpoints. Can be problematic for interpretation. Potential solution: put constraints on polynomials that force them to connect in a continuous/smooth way.

Definition A *polynomial spline of degree d* with *knots* $c_1 < c_2 < ... < c_K$ is a function $g : \mathbb{R} \to \mathbb{R}$ with the following properties

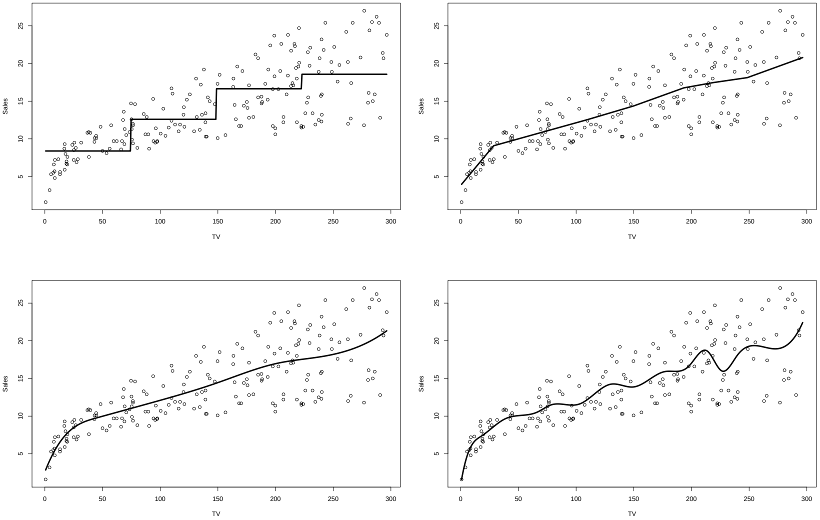
1. On each interval $(-\infty, c_1), [c_1, c_2], ..., [c_K, \infty)$ g is a polynomial of degree $\leq d$ (the degree of a polynomial is the highest power of x with non-zero coefficient).
2. The function g is $d - 1$ times continuously differentiable on $\mathbb{R}$ (0 times continuously differentiable interpreted as continuous).





Example: different methods predicting Sales from TV

Plots below show a spline of degree 3 and 16 df, a step function (which #intervals?), a spline of degree 1 and 8 df and a spline of degree 3 and 8 df. Which one is which?



More on splines

Theorem Any polynomial spline g of degree d with knots $c_0, ..., c_K$ can be represented as

$$g(x) = \beta_1 + ... + \beta_{d+1} x^d + \beta_{d+2}(x - c_1)_+^d + ... + \beta_{K+d+1}(x - c_K)_+^d$$

where

$$(x-c)_+ := \begin{cases} x - c, & x - c > 0 \\ 0, & x - c \leq 0. \end{cases}$$

- Interpretation: fitting polynomial splines with given knots can be achieved by fitting a linear model with transformed predictor x.
- Similar result is true for natural cubic splines.

Popular approach: cubic splines and natural cubic splines.

- *Cubic splines* are splines of degree 3.
- *Natural cubic splines* have the additional constraint that g is linear on $(\infty, c_1]$ and $[c_K, \infty)$.

Degrees of freedom for splines

The *degrees of freedom* of a spline correspond to the number of parameters in a linear model that describes the spline. It depends on the number of knots, the degree of the spline, and on whether the spline is a natural spine. Examples (answers in lectures):

1. A linear spline with 1 knot.

2. A degree d spline with 1 knot.

3. A cubic spline with 3 knots.

Degrees of freedom for splines

The *degrees of freedom* of a spline correspond to the number of parameters in a linear model that describes the spline. It depends on the number of knots, the degree of the spline, and on whether the spline is a natural spine. Examples (answers in lectures):

1. A natural cubic spline with K knots.

Fitting splines in R

One implementation: `splines` package.

Polynomial splines: function **bs(x,...)**. Arguments:

- **degree**: degree of polynomial, values $1, 2, \ldots$
- **knots**: specify knots as discussed in lectures.
- **df**: specify degrees of freedom.
- Only one of the arguments **df** or `knots` should be specified at a time.
- If **df** is specified knots will be put at quantiles of predictors.

Natural cubic splines: function **ns(x,...)**. Arguments:

- **knots**: specify knots as discussed in lectures.
- **df**: specify degrees of freedom.
- Only one of the arguments **df** or **knots** should be specified at a time.
- If **df** is specified knots will be put at quantiles of predictors.

Both functions output a matrix with predictors that can be used in function **lm**.

Counting degrees of freedom: by default, no intercept is included. Thus degrees of freedom are counted without intercept, so 1 less than discussed in lectures.

Basis expansions: closing remarks

All the methods we discussed so far (step functions, piecewise polynomials, polynomial splines, natural splines) can be formulated as liner models with new predictors that are obtained from non-linear transformations of predictor x, i.e. a model of the form

$$f(x) = b_0 + b_1 g_1(x) + b_2 g_2(x) + ... + b_{d+1} g_d(x)$$

where $g_1, ..., g_d$ are fixed functions. This is sometimes called *basis expansion*. We discussed several choices of functions $g_1, ..., g_d$, many more choices exist.

- Some popular approaches we did not cover: wavelets and Fourier series.

- A common problem is all examples above: we need to pick the location of knots/breakpoints.
- Automatic way: pick according to quantiles of predictors, or uniformly spaced over range of predictors. This does not always work well and motivates smoothing splines discussed next.

Exercise: write down the functions $g_1, ..., g_d$ for the following cases: polynomial regression of degree 4, step functions with intervals $(0, 1], (1, 3], (3, 4]$, piecewise polynomial regression of degree 3 with intervals $(0, 2], (2, 4]$, piecewise polynomial regression of degree 1 with intervals $(0, 2], (2, 4]$, cubic spline with knots $1, 2, 5$.

Some R examples:

`Sta314-2021-Lecture07-BasisExpansions.R`

`Sta314-2021-Lecture07-Motorcycle.R`